

ИЗПИТ ПО ФУНКЦИОНАЛНО ПРОГРАМИРАНЕ

КН, 2-ри курс, 2-ри поток (04.02.2023 г.)

Задача 1 (Racket)

Да се дефинира процедура (`get-idxs xs`), която получава един аргумент - списък от числа `xs` и връща анонимна процедура на един аргумент - число `n`. Върнатата процедура трябва да намира двойка от индексите на числа от списъка, чиято сума е равна на `n`. Предполага се, че такива индекси винаги съществуват. Да се връща първата намерена двойка.

Примери:

```
((get-idxs '(2 7 11 15)) 9) → '(0 . 1) ; 2 + 7 = 9
((get-idxs '(3 2 4)) 6)    → '(1 . 2)
((get-idxs '(3 3)) 6)      → '(0 . 1)
```

Задача 2 (Racket или Haskell)

Да се дефинира функция (`sum-arrays xss shift`) (за Haskell: функция `sumArrays :: [[Int]] -> Int -> [Int]`), която получава списък от списъци с цели числа и неотрицателно цяло число `k`. Всички подсписъци имат еднаква дължина $n \geq 1$. `k` е стойност от 0 до `n`, задаваща отместване по следния начин: първият подсписък не се отмества, вторият подсписък се отмества надясно с `k` позиции, третият подсписък се отмества надясно с $2*k$ с позиции, четвъртият подсписък се отмества надясно с $3*k$ с позиции и т.н. При отместването списъците се допълват в началото с нули. Функцията `sum-arrays` (за Haskell: `sumArrays`) трябва да върне нов списък, в който всеки елемент е сума от отместените елементи във всеки подсписък.

Примери (Racket):

```
(sum-arrays '((1 2 3 4 5 6) (7 7 7 7 7 -7)) 0) → '(8 9 10 11 12 -1)
(sum-arrays '((1 2 3 4 5 6) (7 7 7 7 7 7)) 3) → '(1 2 3 11 12 13 7 7 7)
(sum-arrays '((1 2 3 4 5 6) (7 7 7 -7 7 7) (1 1 1 1 1 1)) 3) →
'(1 2 3 11 12 13 -6 8 8 1 1 1)
(sum-arrays '((9 -2 -1 -9 -7 9 -6 -1 9)
              (-7 4 7 2 -7 -8 5 -6 6)
              (2 9 3 -6 -2 -4 2 -6 3)
              (-2 -4 8 1 -3 9 5 7 7)) 3) →
'(9 -2 -1 -16 -3 16 -2 1 4 -3 -12 10 3 -9 12 5 7 7)
(sum-arrays '((-8 -8) (0 -6) (4 -8) (2 -4) (-2 4) (-1 -5) (7 -2)
              (-5 4)) 1) → '(-8 -8 -2 -6 -6 3 2 -7 4)
```

Примери (Haskell):

```
sumArrays [[1,2,3,4,5,6], [7,7,7,7,7,-7]] 0 → [8,9,10,11,12,-1]
sumArrays [[1,2,3,4,5,6], [7,7,7,7,7,7]] 3 →
[1,2,3,11,12,13,7,7,7]
sumArrays [[1,2,3,4,5,6], [7,7,7,-7,7,7], [1,1,1,1,1,1]] 3 →
[1,2,3,11,12,13,-6,8,8,1,1,1]
sumArrays [[9, -2, -1, -9, -7, 9, -6, -1, 9],
           [-7, 4, 7, 2, -7, -8, 5, -6, 6],
           [2, 9, 3, -6, -2, -4, 2, -6, 3],
           [-2, -4, 8, 1, -3, 9, 5, 7, 7]] 3 →
[9,-2,-1,-16,-3,16,-2,1,4,-3,-12,10,3,-9,12,5,7,7]
sumArrays [[-8, -8], [0, -6], [4, -8], [2, -4], [-2, 4], [-1, -5],
           [7, -2], [-5, 4]] 1 → [-8,-8,-2,-6,-6,3,2,-7,4]
```

Обяснения:

В пример 1 отместените подписъци изглеждат така:

```
1,2,3,4,5, 6
7,7,7,7,7,-7
```

В пример 2 отместените подписъци изглеждат така:

```
1,2,3,4,5,6
    7,7,7,7,7,7
```

В пример 3 отместените подписъци изглеждат така:

```
1,2,3,4,5,6
    7,7,7,-7,7,7
        1,1,1,1,1,1
```

Задача 3 (Racket или Haskell)

В един град живеят n души. Нека ги номерираме с етикети от 1 до n . Отскоро се носи слух, че в града има измамник.

За да бъде този слух верен, трябва да са изпълнени следните две условия:

1. Измамникът няма доверие на никого.
2. Всички (с изключение на измамника) имат пряко доверие на измамника.

Да се дефинира процедура (`find-judge n ps`) (за Haskell: функция `findJudge :: Int -> [(Int, Int)] -> Int`), която да намира етикета на измамника. Първият аргумент е броят на хората в града. Вторият аргумент е списък от списък от точкови двойки (за Haskell: списък от двойки). Двойките съдържат данни за това, кой на кого има пряко доверие.

Ако слухът се окаже неверен, то функцията да връща **-1**.

Забележки:

1. Точно един човек (ако съществува) ще отговаря и на двете условия.
2. Всеки човек ще познава поне един друг съгражданин.
3. Доверието не е транзитивно, т.е. ако номер 1 има доверие на номер 2 и номер 2 има доверие на номер 3, то от това **НЕ** следва, че номер 1 има доверие на номер 3.

Примери (Racket):

```
(find-judge 2 '((1 . 2))) → 2
(find-judge 3 '((1 . 3) (2 . 3))) → 3
(find-judge 3 '((1 . 3) (2 . 3) (3 . 1))) → -1
(find-judge 3 '((1 . 2) (2 . 3))) → -1
(find-judge 4 '((1 . 3) (1 . 4) (2 . 3) (2 . 4) (4 . 3))) → 3
```

Примери (Haskell):

```
findJudge 2 [(1, 2)] → 2
findJudge 3 [(1, 3), (2, 3)] → 3
findJudge 3 [(1, 3), (2, 3), (3, 1)] → -1
findJudge 3 [(1, 2), (2, 3)] → -1
findJudge 4 [(1, 3), (1, 4), (2, 3), (2, 4), (4, 3)] → 3
```

Задача 4 (Haskell)

Едно дърво ще наричаме *специално*, ако оценката му спрямо подаден речник от думи е просто число.

Правилата за оценяване на дърво са следните:

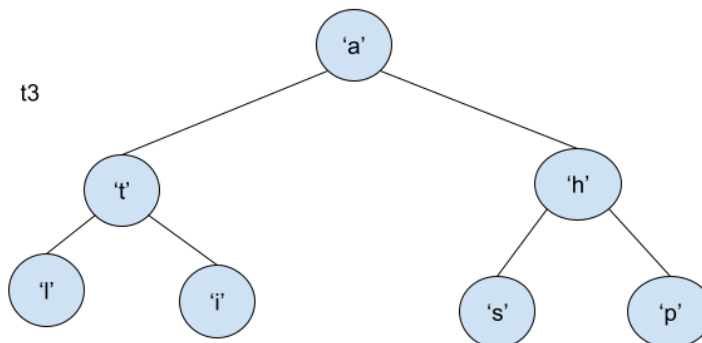
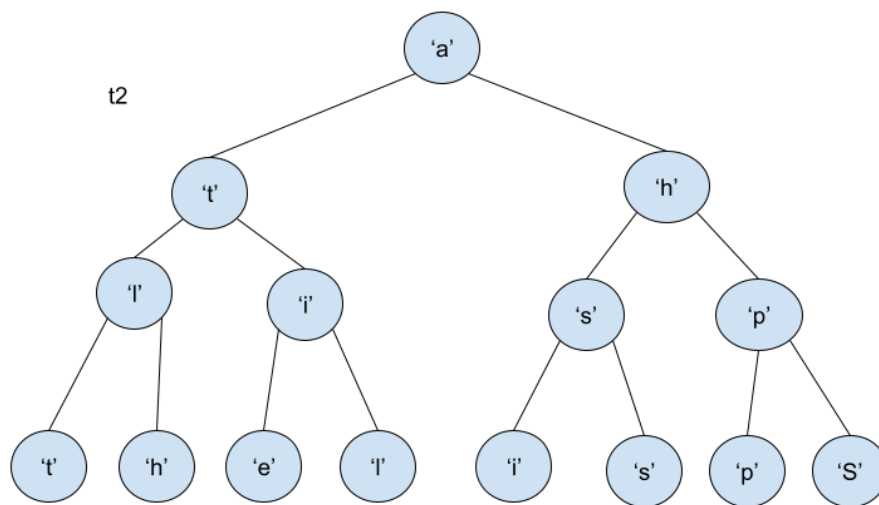
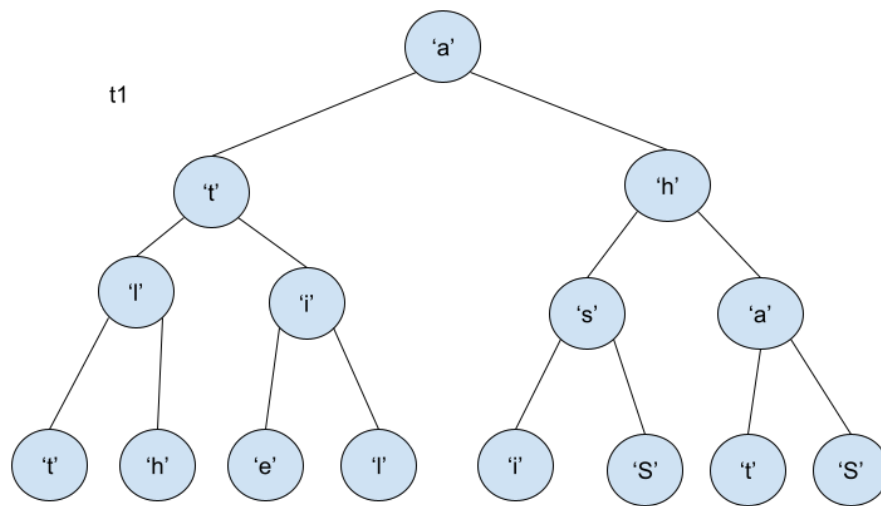
- оценката на дървото е равна на сумата от теглата на отделните думи от речника за всяко от нивата на това дърво;
- ако дума от речника е подниз на низа, образуван при обхождане на върховете (от ляво надясно) на дадено ниво на дървото, теглото на тази дума за това ниво е равно на сумата от дължината на думата и номера на нивото;
- в противен случай теглото на думата за разглежданото ниво е 0.

Да се дефинира функция `isPrimeDictionary :: BTree -> Vocabulary -> Bool`, която получава дърво от символи (знакове) и множество от думи и проверява дали дървото е *специално*.

Примери:

```
type Vocabulary = [String]

data BTree = Nil | Node Char BTree BTree
```



deriving (Show)

```
vocabulary :: Vocabulary
```

```
vocabulary = ["the", "a", "Some", "swimming", "liStS", "lisp"]
```

```
t1 :: BTree
```

```
t1 = Node 'a' (Node 't' (Node 'l' (Node 't' Nil Nil) (Node 'h' Nil Nil)) (Node 'i' (Node 'e' Nil Nil) (Node 'l' Nil Nil))) (Node 'h' (Node 's' (Node 'i' Nil Nil) (Node 'S' Nil Nil)) (Node 'a' (Node 't' Nil Nil) (Node 'S' Nil Nil)))
```

```

t2 :: BTree
t2 = Node 'a' (Node 't' (Node 'l' (Node 't' Nil Nil) (Node 'h' Nil Nil)) (Node 'i' (Node 'e' Nil Nil) (Node 'l' Nil Nil))) (Node 'h' (Node 's' (Node 'i' Nil Nil) (Node 's' Nil Nil)) (Node 'p' (Node 'p' Nil Nil) (Node 'S' Nil Nil)))

t3 :: BTree
t3 = Node 'a' (Node 't' (Node 'l' Nil Nil) (Node 'i' Nil Nil)) (Node 'h' (Node 's' Nil Nil) (Node 'p' Nil Nil))

isPrimeDictionary t1 vocabulary → False
isPrimeDictionary t2 vocabulary → False
isPrimeDictionary t3 vocabulary → True

```

Обяснения:

За t1:

- За това дърво образуваните низове са: "a" (ниво 0), "th" (ниво 1), "lisa" (ниво 2) и "theliStS" (ниво 3).
- Думата "the" се среща един път на ниво 3 и има дължина 3. Текущата оценка е: $3 + 3 = 6$.
- Думата "a" се среща два пъти на ниво 0 и ниво 2 и има дължина 1. Текущата оценка става: $6 + (0 + 1) + (2 + 1) = 10$.
- Думите "Some" и "swimming" не се срещат. Текущата оценка не се променя.
- Думата "liStS" се среща един път на ниво 3 и има дължина 5. Текущата оценка става: $10 + (3 + 5) = 18$.
- Думата "lisp" не се среща. Текущата оценка не се променя.
- 18 не е просто число и следователно функцията връща **False**.

За t3:

- За това дърво образуваните низове са: "a" (ниво 0), "th" (ниво 1) и "lisp" (ниво 2).
- Думата "a" се среща един път на ниво 0 и има дължина 1. Текущата оценка става: $0 + 1 = 1$.
- Думата "lisp" се среща един път на ниво 2 и има дължина 4. Текущата оценка става: $1 + (2 + 4) = 7$.
- 7 е просто число и следователно функцията връща **True**.